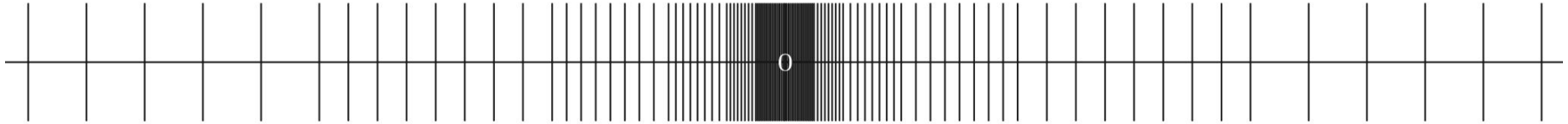
Tools to Detect and Diagnose Floating-Point Errors in Heterogeneous Computing Hardware and Software

A Half-Day Tutorial at Supercomputing 2025

Tut 122: Sunday, 16 November 2025, 8:30am - 12:00pm CST : **Location 121**



Lawrence Livermore National Laboratory

KAHLERT SCHOOL OF COMPUTING
THE UNIVERSITY OF UTAH

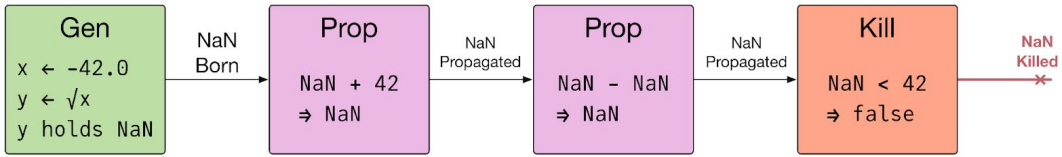
UCDAVIS
UNIVERSITY OF CALIFORNIA

FP Exceptions (Exceptional Values) are Undesirable (*often* are bugs)

- FP32: (1.9×10^{19})^2 → INF … (in E4M3, 16^2 = INF) ⇐ a popular ML low-prec format
  - Easily generated during dot-product
- Of the FP Exceptions, we focus on INF and NaN
  - NaNs can skew control-flow
- This macro is buggy
  - #define MAX(x,y) ((x >= y) ? x : y)

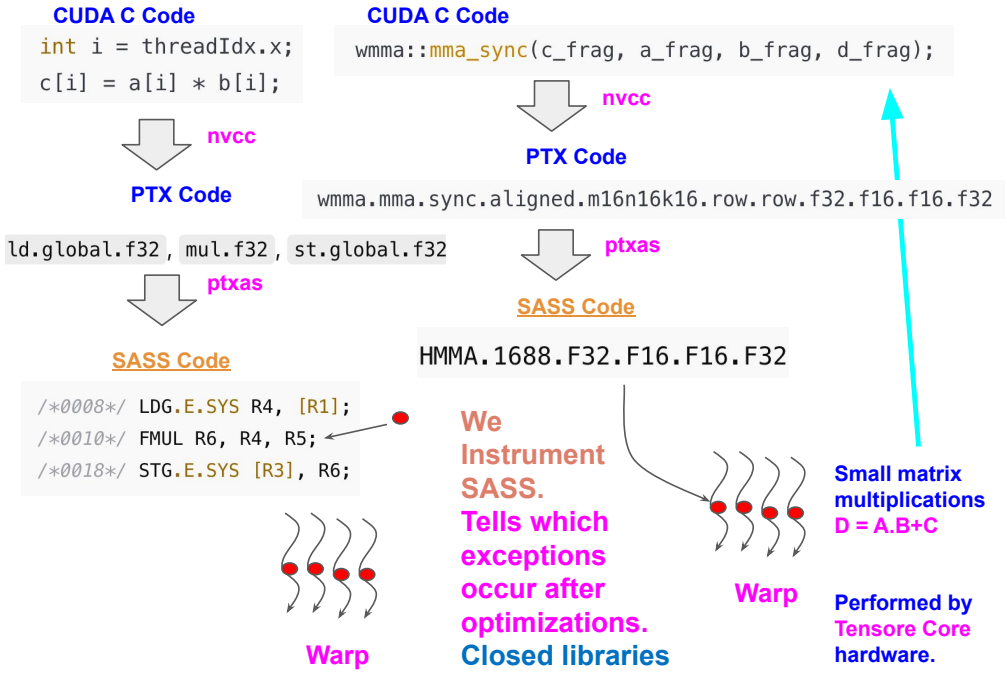
hint : Consider x bound to NaN and y bound to 42; answer is 42. Output "clean"
Then switch these assignments. ; answer is NaN. Max must be commutative

- Exceptions often vanish harmlessly (e.g. in ML codes)
  - But it is difficult to know which control-flows they have affected meanwhile



History: GPU-FPX ⟶ Evolved to become NixNan

First demonstrated during HPDC 2023
Has a detector and an analyzer component that does flow analysis
Exits early when a thread detects exceptions
No support for FP16 or Tensor Cores

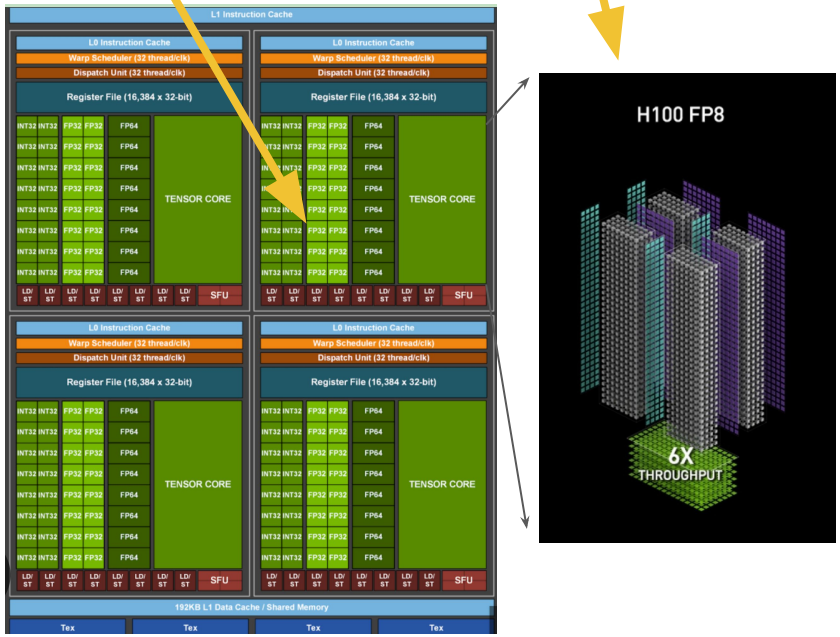


Continuing to refine it in response to tests (discovering missing cases each time…)
Has a single component that detects as well as flow-analyzes
Finds all exceptions (without quitting early)
Support for FP16 and Tensor Cores

NixNan : Detecting Exceptions in NVIDIA GPUs

Detects exceptions in

SIMT Cores as well as Tensor Cores



NixNan : Detecting Exceptions in NVIDIA GPUs

Detects exceptions in

SIMT Cores as well as Tensor Cores



- NVIDIA GPUs do not generate any traps when an FP exception occurs
- Thus, we have to detect exceptions in software
  - By decoding the exponent and mantissa of the computed answer

S	E	M

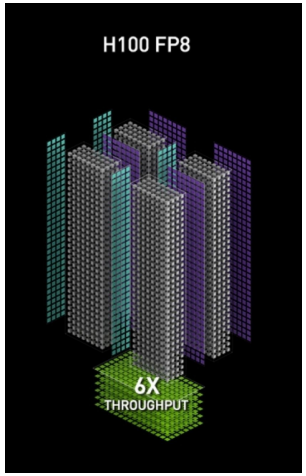
FF	INF (if 0) or NaN (if not)
0	non-zero => subnormal

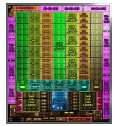
NixNan Detects Exceptions in NVIDIA GPUs per Kernel Launch

Detects exceptions in

SIMT Cores as well as Tensor Cores







> LD_PRELOAD = nixnan.so
> Pre-Compiled Application Binary

Intercept all "Kernel Launch" CUDA Driver API Calls

For Each FP instruction :

Inject_instrumentation();

CUDA Driver APIs

```
#nixnan: ------------ nixnan Report

#nixnan: --- FP16 Operations ---
#nixnan: NaN: 4
#nixnan: Infinity: 3
#nixnan: Subnormal: 2
#nixnan: Division by 0: 0

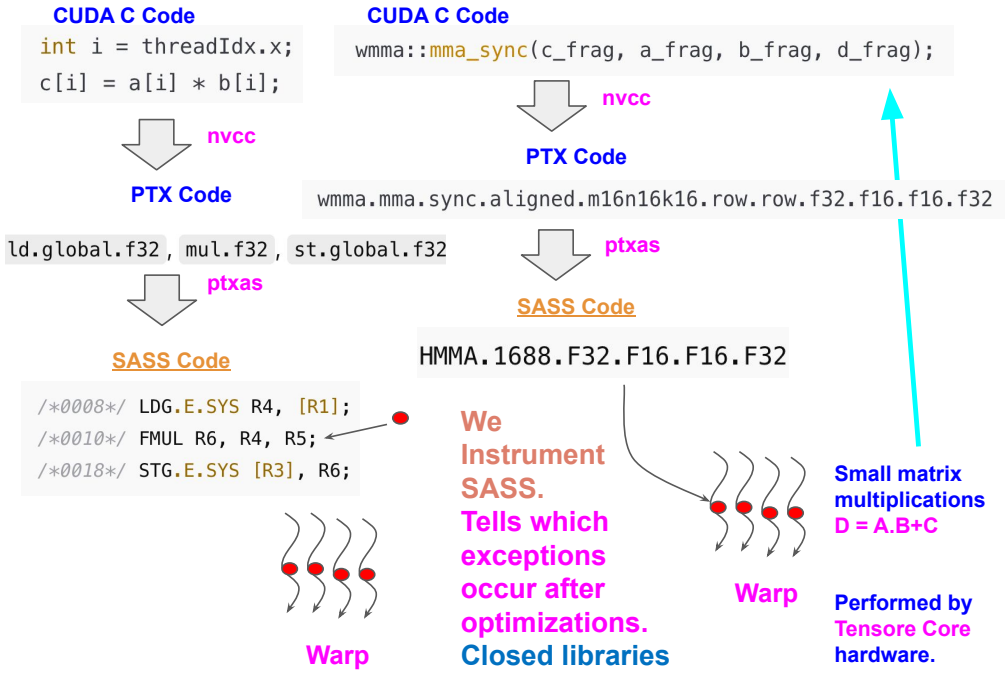
#nixnan: --- FP32 Operations ---
#nixnan: NaN: 0
#nixnan: Infinity: 0
#nixnan: Subnormal: 0
#nixnan: Division by 0: 0

#nixnan: --- FP64 Operations ---
#nixnan: NaN: 0
#nixnan: Infinity: 0
#nixnan: Subnormal: 0
#nixnan: Division by 0: 0
```

```
----- Test NaN: A[0,0]=inf, B[0,0]=1, C[0,0]=-inf -----
A[0,0] = inf (0x7c00), B[0,0] = 1.000000 (0x3c00), C[0,0] = -inf (0xfc00)
Computing D = A * B + C with Tensor Cores...
#nixnan: error [NaN,infinity] detected in instruction HMMA.16816.F16 R20, R4.reuse, R16, RZ ;
#nixnan: error [NaN] detected in instruction HMMA.16816.F16 R22, R4, R18, RZ ; in function WMM
D[0,0]=nan (0x7fff)
```

History: GPU-FPX ⟶ Evolved to become NixNan

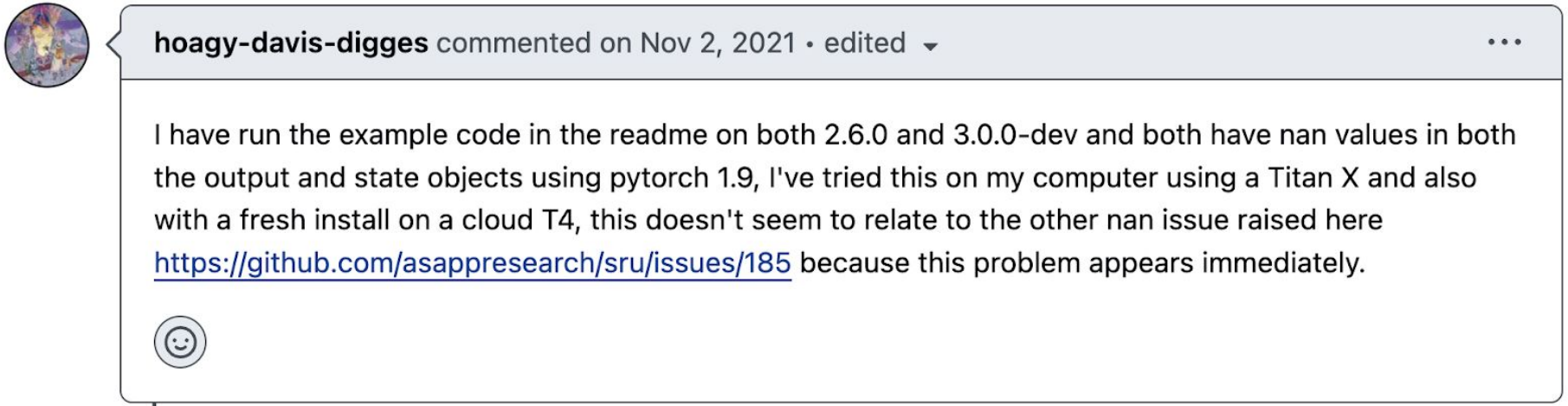
First demonstrated during HPDC 2023
Has a detector and an analyzer component that does flow analysis
Exits early when a thread detects exceptions
No support for FP16 or Tensor Cores



Continuing to refine it in response to tests (discovering missing cases each time…)
Has a single component that detects as well as flow-analyzes
Finds all exceptions (without quitting early)
Support for FP16 and Tensor Cores

Real-world Issue: **NixNan usage**

- Link: https://github.com/asappresearch/sru/issues/193



This case-study was done by Dr. Xinyi Li who developed GPU-FPX, the precursor to NixNan

(the latter is a refined version with bug-fixes, and Tensor-Core Support)

We have adapted Dr. Li's test and fix to NixNan.

Issue

- Link: https://github.com/asappresearch/sru/issues/193

taolei87 commented on Nov 5, 2021 — Contributor

hi **@hoagy-davis-digges** , did you mean you tried the following example and got NaN?

```
import torch
from sru import SRU, SRUCell

# input has length 20, batch size 32 and dimension 128
x = torch.FloatTensor(20, 32, 128).cuda()

input_size, hidden_size = 128, 128

rnn = SRU(input_size, hidden_size,
    num_layers = 2,          # number of stacking RNN layers
    dropout = 0.0,           # dropout applied between RNN layers
    bidirectional = False,   # bidirectional RNN
    layer_norm = False,      # apply layer normalization on the output of each layer
    highway_bias = -2,        # initial bias of highway gate (<= 0)
)
rnn.cuda()

output_states, c_states = rnn(x)      # forward pass
```

Tool Usage

```
1999  python run_sru.py
2000  LD_PRELOAD=~/nixnan.so python run_sru.py
2001  LD_PRELOAD=~/nixnan.so python run_sru_fixed.py
```

Seeing NaNs early, we examined the source-code...

```
#nixnan: running kernel [void at::native::vectorized_elementwise_kernel] ...
#nixnan: running kernel [ampere_sgemm_32x128_nn] ...
#nixnan: error [subnormal] detected in operand 2 of instruction FFMA R1, R32.reu
se, R40.reuse, R1 ; in function ampere_sgemm_32x128_nn of type f32
#nixnan: error [subnormal] detected in operand 0 of instruction FFMA R0, R32, R4
1.reuse, R0 ; in function ampere_sgemm_32x128_nn of type f32
#nixnan: error [subnormal] detected in operand 2 of instruction FFMA R10, R34, R
43.reuse, R10 ; in function ampere_sgemm_32x128_nn of type f32
#nixnan: error [subnormal] detected in operand 2 of instruction FFMA R5, R33.reu
se, R40.reuse, R5 ; in function ampere_sgemm_32x128_nn of type f32
#nixnan: error [subnormal] detected in operand 2 of instruction FFMA R3, R32.reu
se, R42, R3 ; in function ampere_sgemm_32x128_nn of type f32
#nixnan: error [subnormal] detected in operand 0 of instruction FFMA R4, R33, R4
1.reuse, R4 ; in function ampere_sgemm_32x128_nn of type f32
#nixnan: error [subnormal] detected in operand 2 of instruction FFMA R0, R32, R4
1.reuse, R0 ; in function ampere_sgemm_32x128_nn of type f32
#nixnan: error [subnormal] detected in operand 0 of instruction FFMA R1, R36.reu
se, R44.reuse, R1 ; in function ampere_sgemm_32x128_nn of type f32
#nixnan: error [subnormal] detected in operand 3 of instruction FFMA R8, R38.reu
se, R45.reuse, R8 ; in function ampere_sgemm_32x128_nn of type f32
#nixnan: error [NaN,subnormal] detected in operand 0 of instruction FFMA R5, R37
.reuse, R44.reuse, R5 ; in function ampere_sgemm_32x128_nn of type f32
```

Found uninitialized the tensor; setting that fixes NaNs

taolei87 commented on Nov 5, 2021 — Contributor

hi **@hoagy-davis-digges** , did you mean you tried the following example and got NaN?

```
import torch
from sru import SRU, SRUCell

# input has length 20, batch size 32 and dimension 128
x = torch.FloatTensor(20, 32, 128).cuda()

input_size, hidden_size = 128, 128

rnn = SRU(input_size, hidden_size,
    num_layers = 2,          # number of stacking RNN layers
    dropout = 0.0,           # dropout applied between RNN layers
    bidirectional = False,   # bidirectional RNN
    layer_norm = False,      # apply layer normalization on the output of each layer
    highway_bias = -2,        # initial bias of highway gate (<= 0)
)
rnn.cuda()

output_states, c_states = rnn(x)      # forward pass
```

→ torch.randn(20,32,128).cuda()

Exceptions Written Into Memory

Before Fixing SRU

```
#nixnan: --- FP16 Operations ---
#nixnan: NaN:                       0 (0 repeats)
#nixnan: Infinity:                  0 (0 repeats)
#nixnan: -Infinity:                 0 (0 repeats)
#nixnan: Subnormal:                 0 (0 repeats)
#nixnan: Division by 0:             0 (0 repeats)

#nixnan: --- BF16 Operations ---
#nixnan: NaN:                       0 (0 repeats)
#nixnan: Infinity:                  0 (0 repeats)
#nixnan: -Infinity:                 0 (0 repeats)
#nixnan: Subnormal:                 0 (0 repeats)
#nixnan: Division by 0:             0 (0 repeats)

#nixnan: --- FP32 Operations ---
#nixnan: NaN:                     843 (2418829 repeats)
#nixnan: Infinity:                 13 (15635 repeats)
#nixnan: -Infinity:                 0 (0 repeats)
#nixnan: Subnormal:               746 (251293 repeats)
#nixnan: Division by 0:             0 (0 repeats)

#nixnan: --- FP64 Operations ---
#nixnan: NaN:                       0 (0 repeats)
#nixnan: Infinity:                  0 (0 repeats)
#nixnan: -Infinity:                 0 (0 repeats)
#nixnan: Subnormal:                 0 (0 repeats)
#nixnan: Division by 0:             0 (0 repeats)

#nixnan: --- FP16 Memory  Operations ---
#nixnan: NaN:                       0 (0 repeats)
#nixnan: --- BF16 Memory  Operations ---
#nixnan: NaN:                       0 (0 repeats)
#nixnan: --- FP32 Memory  Operations ---
#nixnan: NaN:                       3 (5373 repeats)
#nixnan: --- FP64 Memory  Operations ---
#nixnan: NaN:                       0 (0 repeats)
```

Our fix was applied

After Fixing SRU

```
#nixnan: --- FP16 Operations ---
#nixnan: NaN:                       0 (0 repeats)
#nixnan: Infinity:                  0 (0 repeats)
#nixnan: -Infinity:                 0 (0 repeats)
#nixnan: Subnormal:                 0 (0 repeats)
#nixnan: Division by 0:             0 (0 repeats)

#nixnan: --- BF16 Operations ---
#nixnan: NaN:                       0 (0 repeats)
#nixnan: Infinity:                  0 (0 repeats)
#nixnan: -Infinity:                 0 (0 repeats)
#nixnan: Subnormal:                 0 (0 repeats)
#nixnan: Division by 0:             0 (0 repeats)

#nixnan: --- FP32 Operations ---
#nixnan: NaN:                       0 (0 repeats)
#nixnan: Infinity:                  0 (0 repeats)
#nixnan: -Infinity:                 0 (0 repeats)
#nixnan: Subnormal:                 0 (0 repeats)
#nixnan: Division by 0:             0 (0 repeats)

#nixnan: --- FP64 Operations ---
#nixnan: NaN:                       0 (0 repeats)
#nixnan: Infinity:                  0 (0 repeats)
#nixnan: -Infinity:                 0 (0 repeats)
#nixnan: Subnormal:                 0 (0 repeats)
#nixnan: Division by 0:             0 (0 repeats)

#nixnan: --- FP16 Memory  Operations ---
#nixnan: NaN:                       0 (0 repeats)
#nixnan: --- BF16 Memory  Operations ---
#nixnan: NaN:                       0 (0 repeats)
#nixnan: --- FP32 Memory  Operations ---
#nixnan: NaN:                       0 (0 repeats)
#nixnan: --- FP64 Memory  Operations ---
#nixnan: NaN:                       0 (0 repeats)
```

FP Exceptions in HPC : Ill-conditioned Matrix

```
#nixnan: ------------ nixnan Report -----------

#nixnan: --- FP16 Operations ---
#nixnan: NaN:                      0 (0 repeats)
#nixnan: Infinity:                 0 (0 repeats)
#nixnan: -Infinity:                0 (0 repeats)
#nixnan: Subnormal:                0 (0 repeats)
#nixnan: Division by 0:            0 (0 repeats)

#nixnan: --- BF16 Operations ---
#nixnan: NaN:                      0 (0 repeats)
#nixnan: Infinity:                 0 (0 repeats)
#nixnan: -Infinity:                0 (0 repeats)
#nixnan: Subnormal:                0 (0 repeats)
#nixnan: Division by 0:            0 (0 repeats)

#nixnan: --- FP32 Operations ---
#nixnan: NaN:                      0 (0 repeats)
#nixnan: Infinity:                 0 (0 repeats)
#nixnan: -Infinity:                0 (0 repeats)
#nixnan: Subnormal:                0 (0 repeats)
#nixnan: Division by 0:            0 (0 repeats)

#nixnan: --- FP64 Operations ---
#nixnan: NaN:                      0 (0 repeats)
#nixnan: Infinity:                 0 (0 repeats)
#nixnan: -Infinity:                0 (0 repeats)
#nixnan: Subnormal:                0 (0 repeats)
#nixnan: Division by 0:            0 (0 repeats)

#nixnan: --- FP16 Memory  Operations ---
#nixnan: NaN:                      0 (0 repeats)
#nixnan: --- BF16 Memory  Operations ---
#nixnan: NaN:                      0 (0 repeats)
#nixnan: --- FP32 Memory  Operations ---
#nixnan: NaN:                      0 (0 repeats)
#nixnan: --- FP64 Memory  Operations ---
#nixnan: NaN:                      0 (0 repeats)
```

```
--- Solution X Vector (10)
1.12338
0.799707
7.82785
-1954.76
-0.000778511
1.07278
-0.535868
-3.61902
8.83203
0.585185
```

LD_PRELOAD=~/nixnan.so
CUDA_MATRIX=matrix_0.csv ./LU_solver

LD_PRELOAD=~/nixnan.so
CUDA_MATRIX=matrix_5.csv ./LU_solver

```
--- Solution X Vector (10)
1.4665
inf
2.15945
0.895458
10.9171
1.11917
inf
24.0914
0.421027
-1.39342
```

```
#nixnan: ------------ nixnan Report -----------

#nixnan: --- FP16 Operations ---
#nixnan: NaN:                      0 (0 repeats)
#nixnan: Infinity:                 0 (0 repeats)
#nixnan: -Infinity:                0 (0 repeats)
#nixnan: Subnormal:                0 (0 repeats)
#nixnan: Division by 0:            0 (0 repeats)

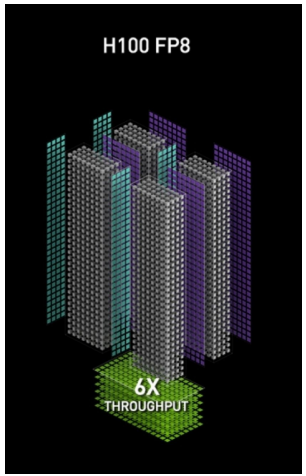
#nixnan: --- BF16 Operations ---
#nixnan: NaN:                      0 (0 repeats)
#nixnan: Infinity:                 0 (0 repeats)
#nixnan: -Infinity:                0 (0 repeats)
#nixnan: Subnormal:                0 (0 repeats)
#nixnan: Division by 0:            0 (0 repeats)

#nixnan: --- FP32 Operations ---
#nixnan: NaN:                      4 (0 repeats)
#nixnan: Infinity:                 0 (0 repeats)
#nixnan: -Infinity:                0 (0 repeats)
#nixnan: Subnormal:                0 (0 repeats)
#nixnan: Division by 0:            0 (0 repeats)

#nixnan: --- FP64 Operations ---
#nixnan: NaN:                     48 (0 repeats)
#nixnan: Infinity:                 2 (0 repeats)
#nixnan: -Infinity:                0 (0 repeats)
#nixnan: Subnormal:                0 (0 repeats)
#nixnan: Division by 0:            2 (2 repeats)

#nixnan: --- FP16 Memory  Operations ---
#nixnan: NaN:                      0 (0 repeats)
#nixnan: --- BF16 Memory  Operations ---
#nixnan: NaN:                      0 (0 repeats)
#nixnan: --- FP32 Memory  Operations ---
#nixnan: NaN:                      0 (0 repeats)
#nixnan: --- FP64 Memory  Operations ---
#nixnan: NaN:                      0 (0 repeats)
```

Deep-Dive : Tensor-Core Support of NixNan



> LD_PRELOAD = nixnan.so
> Pre-Compiled Application Binary

Intercept all "Kernel Launch" CUDA Driver API Calls

For Each FP instruction :

Inject_instrumentation();

CUDA Driver APIs

```
----- Test NaN: A[0,0]=inf, B[0,0]=1, C[0,0]=-inf -----
A[0,0] = inf (0x7c00), B[0,0] = 1.000000 (0x3c00), C[0,0] = -inf (0xfc00)
Computing D = A * B + C with Tensor Cores...
#nixnan: error [NaN,infinity] detected in instruction HMMA.16816.F16 R20, R4.reuse, R16, RZ ;
#nixnan: error [NaN] detected in instruction HMMA.16816.F16 R22, R4, R18, RZ ; in function WMM
D[0,0]=nan (0x7fff)
```

Tensor core register layout

Each thread holds part of the matrices in its registers

T1

R0	R1	R2	R3	R4	R5	R6	R7
A0	*A1*	*A2*	*A3*	*B0*	*B1*	*C0*	*C1*

T2

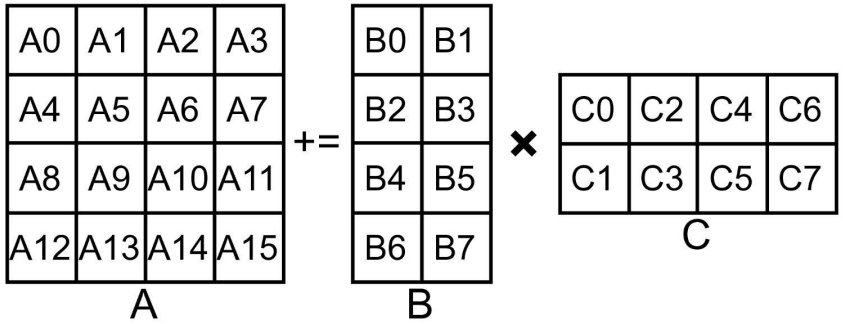
R0	R1	R2	R3	R4	R5	R6	R7
A4	*A5*	*A6*	*A7*	*B2*	*B3*	*C2*	*C3*

T3

R0	R1	R2	R3	R4	R5	R6	R7
A8	*A9*	*A10*	*A11*	*B4*	*B5*	*C4*	*C5*

T4

R0	R1	R2	R3	R4	R5	R6	R7
A12	*A13*	*A14*	*A15*	*B6*	*B7*	*C6*	*C7*

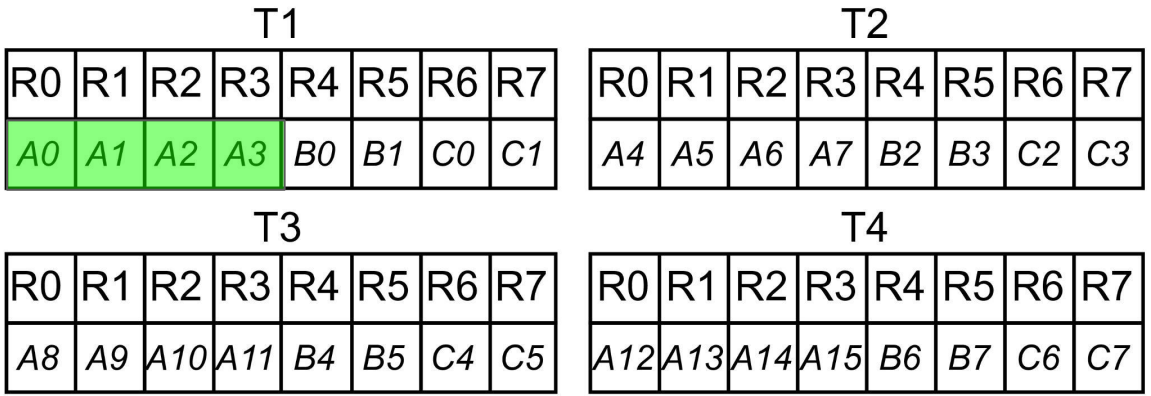


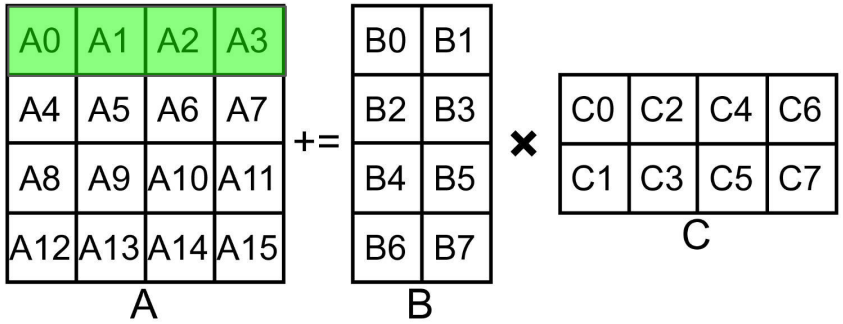
Tensor core register layout

Each thread holds part of the matrices in its registers

Thread T1 holds:

- The first four elements of A



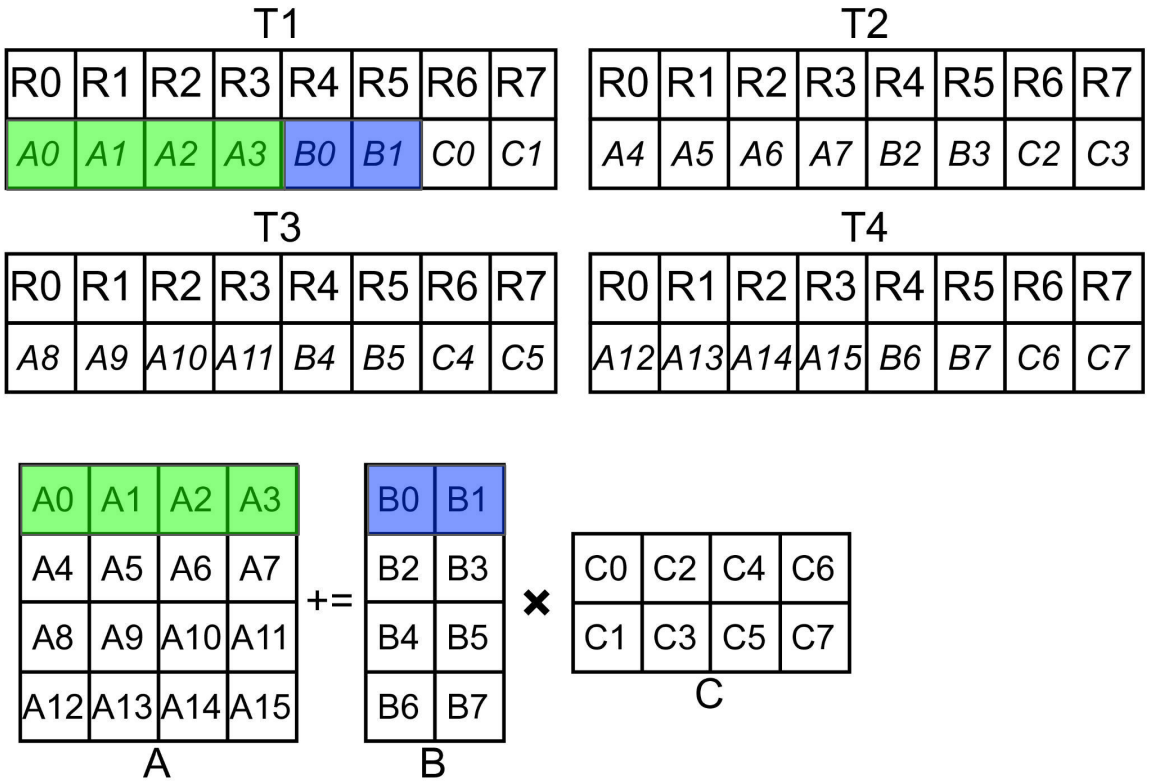


Tensor core register layout

Each thread holds part of the matrices in its registers

Thread T1 holds:

- The first four elements of A
- The first two elements of B



Tensor core register layout

Each thread holds part of the matrices in its registers

Thread T1 holds:

- The first four elements of A
- The first two elements of B
- The first two elements of C

